Compte rendu – TP Déploiement d’un modèle avec FastAPI

Groupe / étudiants : À compléter

Modèle : Fashion-MNIST – réseau MLP fourni (fashion_mnist.keras)

1. Objectif et contrat d’inférence

L’application expose un service HTTP permettant de classer une image Fashion-MNIST de 28 × 28 pixels. La route POST /predict reçoit un objet JSON contenant une liste pixels de 784 valeurs. Elle renvoie l’identifiant et le nom de la classe, la confiance et la latence d’inférence. Les dix classes sont : T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot.

2. Résultats des 20 prédictions

Image	Classe réelle	Classe prédite	Confiance	Latence (ms)	Résultat
0	Ankle boot	Ankle boot	0.9652	822.494	Correct
1	Pullover	Pullover	0.9904	5.588	Correct
2	Trouser	Trouser	1.0000	5.103	Correct
3	Trouser	Trouser	0.9999	4.822	Correct
4	Shirt	Shirt	0.6753	4.462	Correct
5	Trouser	Trouser	0.9995	5.762	Correct
6	Coat	Coat	0.9946	5.294	Correct
7	Shirt	Shirt	0.9882	7.869	Correct
8	Sandal	Sandal	1.0000	7.659	Correct
9	Sneaker	Sneaker	1.0000	6.796	Correct
10	Coat	Coat	0.9820	8.258	Correct
11	Sandal	Sandal	0.9998	5.824	Correct
12	Sneaker	Sandal	0.9347	12.081	Incorrect
13	Dress	Dress	0.9999	10.391	Correct
14	Coat	Coat	0.8230	8.809	Correct
15	Trouser	Trouser	0.9992	9.012	Correct
16	Pullover	Pullover	0.9819	4.710	Correct
17	Coat	Pullover	0.6880	5.629	Incorrect
18	Bag	Bag	1.0000	4.314	Correct
19	T-shirt/top	T-shirt/top	0.9858	6.394	Correct

Prédictions correctes : 18/20

Taux de réussite : 90.00 %

Latence moyenne : 47.564 ms

3. Test d’une entrée incorrecte

Requête envoyée avec 3 pixels. Code HTTP obtenu : 400. Réponse : {'detail': "L'entrée doit contenir exactement 784 pixels."}

4. Visualisation des images testées



5. Réponses aux questions d’analyse

1. Pourquoi le modèle reçoit-il une entrée de forme (1, 28, 28) ?

Le 1 représente le nombre d’images du batch. Les dimensions 28 × 28 représentent la hauteur et la largeur de l’image Fashion-MNIST.

2. Pourquoi l’API vérifie-t-elle la présence de 784 pixels ?

Une image Fashion-MNIST contient 28 × 28 = 784 pixels. Cette vérification garantit que l’image peut être reconstruite correctement.

3. Pourquoi faut-il diviser les pixels par 255 lorsque leur valeur maximale dépasse 1 ?

Les pixels d’origine sont codés entre 0 et 255. La division par 255 les ramène entre 0 et 1, comme lors du prétraitement utilisé pour entraîner le modèle.

4. Quelle différence existe entre les logits et les probabilités ?

Les logits sont les scores bruts produits par la dernière couche du réseau. Les probabilités sont des valeurs normalisées entre 0 et 1 dont la somme vaut 1.

5. Pourquoi applique-t-on une fonction softmax ?

Softmax transforme les logits en probabilités interprétables pour les dix classes. La classe ayant la probabilité la plus élevée est retenue.

6. Pourquoi le modèle est-il chargé une seule fois au démarrage du serveur ?

Pour éviter de recharger le fichier à chaque requête, réduire le temps de traitement et réutiliser le modèle en mémoire.

7. Quelle différence existe entre model.fit() et model(image, training=False) ?

model.fit() entraîne le réseau en ajustant ses paramètres. model(image, training=False) utilise le modèle déjà entraîné pour effectuer une inférence sans apprentissage.

8. Que représente la latence retournée par l’API ?

Elle représente le temps mesuré pour l’appel d’inférence du modèle, en millisecondes. Elle peut varier selon la machine et la charge.

9. Quel est le rôle de la route /health ?

Elle permet de vérifier que l’API répond et que le modèle est chargé, ainsi que d’afficher ses formes d’entrée et de sortie.

10. Quels éléments constituent le contrat d’inférence de cette application ?

Une requête POST /predict contenant pixels (exactement 784 nombres finis entre 0 et 255). L’API reconstruit et normalise l’image puis renvoie class_id, class_name, confidence et latency_ms. Une entrée invalide est refusée avec une erreur explicite.

6. Conclusion

Ce TP a permis de déployer le modèle Fashion-MNIST sous la forme d’une API FastAPI. Le service reçoit une image représentée par 784 pixels, vérifie et prépare les données, puis exécute le modèle pré-entraîné. La fonction Softmax permet d’obtenir un score de confiance pour chacune des dix classes. Les tests sur vingt images permettent de mesurer le taux de réussite et la latence moyenne. Le test avec trois pixels vérifie également que l’API refuse une entrée non conforme. Les résultats numériques présentés dans ce rapport sont générés automatiquement à partir de l’exécution réelle de test_api.py.

Captures à ajouter : placer swagger.png, health.png et prediction.png dans le dossier captures/.